An End-to-End RISC-V-Controlled INT8 CNN Accelerator on a Tang Nano 20K FPGA

Nguyen Phu Huu, Nguyen Trung Nguyen, and Trinh Ngoc Tin

***Abstract*—This paper presents the design, implementation, and evidence-based evaluation of a complete handwritten-digit recognition system on a Tang Nano 20K field-programmable gate array (FPGA). The platform integrates a team-developed RV32I processor, a memory-mapped INT8 convolution accelerator, firmware, UART bootloading, a versioned microSD data format, and HDMI output. A LeNet-inspired network is trained on MNIST and exported through a reproducible floating-point-to-integer workflow. The best recorded Float32 checkpoint and the simulated symmetric INT8 pipeline both achieve 99.50% accuracy on the 10,000-image MNIST test set. A separate balanced set of 100 photographed digits reaches 98/100 accuracy in the host integer-reference pipeline; this result is deliberately not reported as board-wide accuracy. Element-wise golden checks validate both convolution layers against PyTorch, while a full-system ModelSim run exercises the processor, UART loader, SD-backed firmware, and accelerator without simulation errors. Post-place-and-route results show 76% logic, 100% block SRAM, and 92% DSP utilization. Setup timing passes, but 15 hold-violating endpoints remain, with a worst hold slack of -1.435 ns. Relative to the SystemC/TLM virtual-platform study of Lim *et al.*, this work advances from design-space exploration to synthesizable RTL and board integration while explicitly separating model-level, integer-reference, RTL, implementation, and board-observation evidence.**

***Index Terms*—CNN accelerator, RISC-V, RV32I, FPGA, INT8 quantization, hardware-software co-design, RTL verification, Tang Nano 20K, MNIST.**

I. Introduction

EDGE inference requires more than an accurate neural network. A deployable embedded system must move parameters and activations through bounded on-chip storage, preserve numerical behavior after quantization, expose a stable software interface, and satisfy implementation timing. On a small FPGA, these concerns are coupled: a change in buffering or arithmetic can alter both model fidelity and routability [1], [2].

Lim *et al.* [3] studied this design space through a RISC-V virtual platform written in SystemC/TLM 2.0. Their environment enabled rapid exploration of processing-element (PE) count, buffer capacity, precision, and representative Darknet/YOLO-tiny-v3 layers. The study is valuable at the electronic-system level (ESL), but its evidence is primarily simulated data movement, execution behavior, and output similarity. It does not establish synthesis cost, physical timing, a firmware update path, or board-level integration.

This work addresses the complementary question: how can a RISC-V-controlled CNN pipeline be carried from model training to a synthesizable and observable FPGA system while retaining traceable numerical evidence? The implemented platform combines a team-developed RV32I core conforming to the base unprivileged instruction set [4], a fixed 4-by-4 PE convolution engine, firmware-executed pooling and fully connected layers, UART loading, microSD storage, cyclic redundancy checks (CRCs), and HDMI visualization. The workload is handwritten-digit recognition rather than the detection layers used in [3]; therefore, direct claims of speed or accuracy superiority would be invalid.

The contributions are fourfold. First, the work defines a reproducible path from a PyTorch checkpoint to symmetric INT8 tensors and INT32 golden accumulations. Second, it realizes an end-to-end hardware-software system in synthesizable RTL. Third, it evaluates the system through distinct evidence layers: model accuracy, integer-reference behavior, golden arithmetic, full-system simulation, post-place-and-route (P&R) reports, and board observations. Fourth, it reports negative evidence—notably incomplete board-wide measurement and remaining hold violations—instead of treating a visual demonstration as a substitute for controlled evaluation.

II. Related Work

A. CNN Dataflows and FPGA Acceleration

Convolution dominates the multiply-accumulate (MAC) workload of compact vision networks. Accelerator efficiency therefore depends on reusing activations and weights while limiting transfers to slower memory [1]. Architectures such as Eyeriss demonstrate that the dataflow and memory hierarchy can matter as much as arithmetic parallelism [2]. On small FPGAs, PE count cannot be selected independently of block SRAM, DSP capacity, routing pressure, and software overhead.

For an input of size $H \times W$, kernel $K_h \times K_w$, padding P, and stride S, the output dimensions are

$$H_o = \left\lfloor \frac{H + 2P - K_h}{S} \right\rfloor + 1, \quad W_o = \left\lfloor \frac{W + 2P - K_w}{S} \right\rfloor + 1. \quad (1)$$

The idealized layer workload is

$$N_{\mathrm{MAC}} = H_o W_o C_i C_o K_h K_w. \quad (2)$$

The authors are with the Faculty of High Quality Training, Ho Chi Minh City University of Technology and Education, Ho Chi Minh City, Vietnam. Advisor: Huynh Hoang Ha.

Equation (2) excludes address generation, tiling, register access, and firmware stages; these must be measured separately in an implemented system.

B. RISC-V Virtual-Platform Exploration

The reference design couples an RV32IM virtual processor to a deep-learning accelerator through TLM transactions [3]. Its accelerator contains a special-function-register interface, a data loader, convolution and activation pipelines, and five buffers. Representative Darknet/YOLO-tiny-v3 layers are used to explore PE count, 1–8 MB buffers, and 8- versus 32-bit precision. Reported cosine similarities are approximately 85% or higher, and the reported 7.62–107.91 s values are simulator wall-clock times rather than physical inference latency. The abstraction supports broad exploration but intentionally omits synthesis, static timing, physical resources, and a board communication stack.

C. Integer-Only Inference and Verification

Integer inference reduces storage and maps efficiently to FPGA DSP resources, but it introduces scale, rounding, clipping, accumulator-width, and tensor-layout hazards [5]. For a floating-point tensor x, this work uses signed symmetric per-tensor quantization:

$$s_x = \frac{\max |x|}{127}, \quad q = \mathrm{clip}\left(\mathrm{round}\left(\frac{x}{s_x}\right), -127, 127\right). \tag{3}$$

With zero-point equal to zero, convolution accumulates in INT32:

$$a_{o,h,w} = \sum_{c,i,j} q_x(c, h+i, w+j) q_w(o, c, i, j). \tag{4}$$

The real-valued estimate is $(s_x s_w)a$. Post-quantization accuracy is necessary but insufficient; exported byte order, signedness, padding, loop nesting, and accumulator behavior must also match the RTL.

III. Methodology

A. System Architecture

Fig. 1 summarizes the platform. The RV32I core executes boot, data management, pooling, fully connected inference, and result reporting. The accelerator is accessed through memory-mapped I/O (MMIO). Model and image artifacts are loaded through UART or stored in a versioned microSD image. HDMI and the host interface provide observability but are not treated as quantitative instruments.

B. Network and Training

The LeNet-inspired model contains two 3-by-3 convolutions without bias, ReLU nonlinearities [6], 2-by-2 max pooling, a 128-unit fully connected layer with training-time dropout [7], and a ten-class output. Table I gives the tensor progression. The network has 206,874 trainable parameters.

Training uses the 60,000-image MNIST training set and official 10,000-image test set [8], [9]. The random seed is 42, batch size 64, and training lasts 15 epochs. AdamW starts at 10^{-3} with weight decay 10^{-4}; cosine annealing reduces the learning rate to 10^{-5}. Random affine augmentation permits 7 degrees of rotation, 0.08 translation, and 0.95–1.05 scaling.

TABLE I
Network Architecture and Compute Mapping

Stage	Input	Output	Executor	MACs
Conv1+ReLU	1×28×28	16×28×28	FPGA	112,896
MaxPool1	16×28×28	16×14×14	Firmware	–
Conv2+ReLU	16×14×14	32×14×14	FPGA	903,168
MaxPool2	32×14×14	32×7×7	Firmware	–
FC1+ReLU	1,568	128	Firmware	200,704
FC2+argmax	128	10	Firmware	1,280

C. Export and Golden-Model Bridge

The checkpoint, scales, quantized arrays, dimensions, and INT32 golden outputs are generated together. Weights follow output-channel, input-channel, kernel-row, kernel-column order. Activations are signed INT8 and partial sums remain signed INT32. A common sample compares the PyTorch convolution [10], host integer implementation, and RTL-visible buffer element by element. This bridge is stronger than checking only top-1 prediction because layout defects can remain hidden when the classification margin is large.

D. Image Preprocessing

The real-digit set contains 100 photographs, balanced at ten per class. Inputs are converted to grayscale, auto-contrasted, blurred, thresholded, polarity-corrected, cropped, padded, centered, and resized to 28-by-28 before integer conversion. Fig. 2 shows representative transformations.

E. Accelerator and Hardware-Software Partition

The convolution engine contains a controller, address generators, input and weight buffers, an output buffer, and a 4-by-4 PE array. Each PE multiplies signed 8-bit operands and accumulates into a signed 32-bit register. Sixteen PEs process selected output channels or positions in parallel. Weights are held for the inner activation traversal when possible; the controller handles boundaries and partial tiles.

The concrete buffers hold 3,136 input bytes, 4,608 weight bytes, and 12,544 INT32 output words. These capacities are an implementation choice, not the 1–8 MB sweep in [3]. The processor configures dimensions and addresses through MMIO, launches the engine, waits for completion, and executes pooling and fully connected stages in firmware. Approximately 83.4% of network MACs are assigned to the accelerator. Convolution throughput alone therefore cannot represent end-to-end latency.

F. Boot, Storage, and Observability

The UART loader supports packetized writes to instruction memory, data memory, and MMIO registers, with CRC32 protection. Separate AI Runtime and Dataset Manager firmware images decouple execution from data maintenance.

The raw sector-based RCN1 format, version 3, records layout, checksums, and validity metadata. The dataset begins at sector 408 and contains 100 records of 816 bytes; the image occupies 568 sectors, or 290,816 bytes. Updates use read-back and validate-before-commit so that firmware does not consume

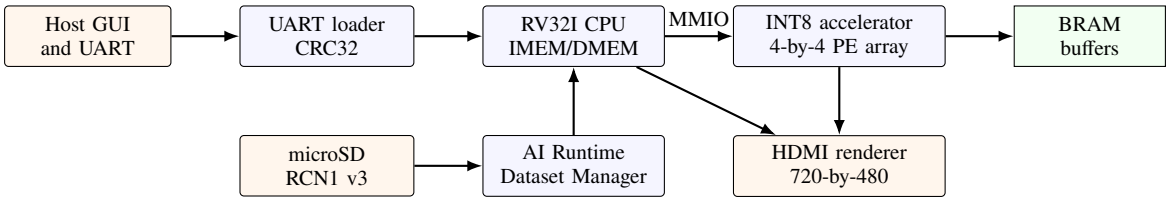


Fig. 1. End-to-end organization of the RISC-V-controlled CNN system. Quantitative conclusions are drawn from the evidence source appropriate to each subsystem.

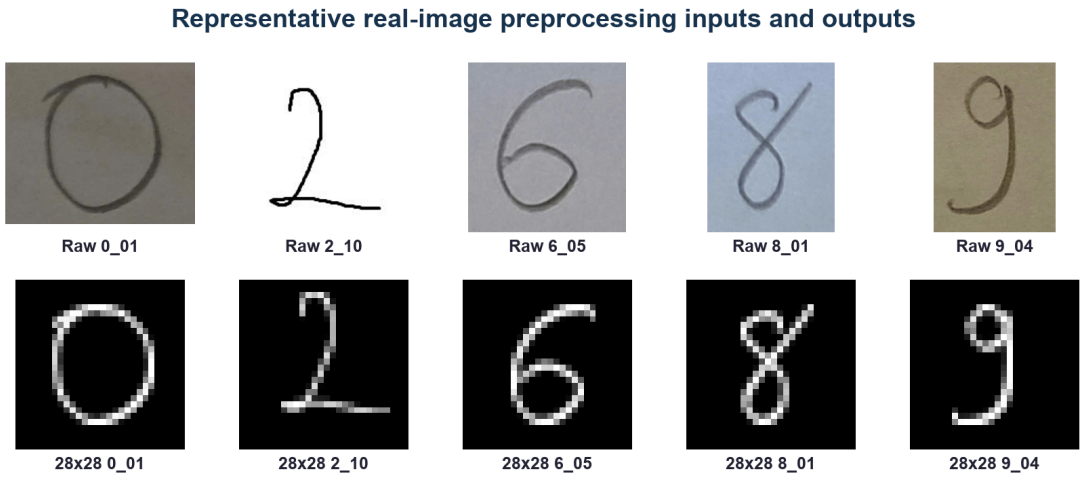


Fig. 2. Representative real-digit inputs and normalized 28-by-28 forms. The balanced set is an engineering regression set, not a population-scale handwriting benchmark.

a partially written record. This is stronger than unchecked writes, although it is not a power-loss-safe journal.

The HDMI path renders a 720-by-480 interface without a full framebuffer and displays the input, prediction, and confidence. The host GUI manages COM connectivity, firmware loading, SD-image creation, and monitoring.

IV. Experimental Setup

A. Evidence Hierarchy and Metrics

Evaluation is separated into five levels. Model evidence covers training and Float32 accuracy. Integer-reference evidence covers simulated INT8 arithmetic and the 100-image set. RTL evidence covers element-wise module checks and the full-system transcript. Implementation evidence comes from final Gowin P&R reports. Board evidence consists of UART and HDMI observations. This hierarchy prevents a result from being generalized beyond the layer at which it was measured.

Accuracy is $N_{\text{correct}}/N_{\text{samples}}$. Quantization loss is the percentage-point difference between Float32 and simulated INT8 accuracy. Resource and timing values are taken directly from final implementation reports. Simulator wall-clock duration is separated from simulated time and from unmeasured board latency.

B. Reproducibility Controls

Model runs use the recorded seed and hyperparameters. Exported tensors carry shapes and scales; SD records include CRC and validity fields. Module tests compare complete output arrays rather than selected entries. Full-system simulation uses the integrated processor, memory, storage, firmware, and accelerator hierarchy. The remaining reproducibility gap is the absence of a version-matched 100-sample board log bound to one artifact manifest.

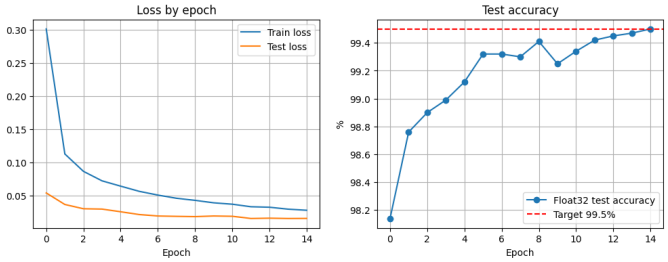


Fig. 3. Recorded loss and test accuracy over 15 epochs.

V. Results and Discussion

A. Training and Quantization

Fig. 3 shows the recorded traces. Training loss decreases from 0.3010 to 0.0280 and test loss from 0.0450 to 0.0157. Test accuracy rises from 98.14% to 99.50%. Because accuracy is not strictly monotonic under augmentation and a changing learning rate, selecting the best recorded checkpoint is more defensible than assuming the final epoch is optimal.

Both Float32 and symmetric per-tensor INT8 achieve 99.50% on 10,000 MNIST test images, a measured loss of

TABLE II
METHODOLOGICAL RELATIONSHIP TO LIM *ET AL.* [3]

Aspect	Lim *et al.*	This work	Technical implication
Abstraction	SystemC/TLM 2.0 virtual platform	Synthesizable Verilog RTL and FPGA P&R	Adds physical resource and timing evidence but reduces exploration flexibility
Processor	Virtual RV32IM	Team-developed RV32I	Removes the M extension and exposes real firmware/accelerator constraints
Workload	Darknet/YOLO-tiny-v3 layers	LeNet-inspired MNIST classifier	Enables an end-to-end application but prevents direct accuracy comparison
Accelerator	Configurable PE count and buffer sweep	Fixed 4-by-4 PE array and bounded BRAM	Provides one implementation point, not a global optimum
System support	ELF/Darknet virtual execution	UART, firmware, versioned microSD, CRC, GUI, and HDMI	Extends the accelerator into an observable system
Verification	Layer similarity and simulated behavior	Golden INT32, RTL, full-system simulation, P&R, and board observations	Broadens evidence and reveals implementation-specific faults

TABLE III
EVALUATION QUESTIONS AND EVIDENCE

Question	Primary evidence
Model quality	10,000-image MNIST test set
Quantization loss	Float32 versus simulated INT8
Export correctness	Conv1/Conv2 INT32 golden arrays
System integration	CPU/UART/SD/accelerator transcript
Physical feasibility	Gowin resource and timing reports
Operational visibility	UART GUI and HDMI photographs

TABLE IV
MODEL-LEVEL ACCURACY

Configuration	Samples	Accuracy
Float32 checkpoint	10,000	99.50%
Simulated INT8	10,000	99.50%

0.00 percentage points. This equality means the selected scales preserve top-1 decisions on this set; it does not imply that every activation equals its Float32 counterpart. MNIST's high contrast and often-large decision margins plausibly explain the low sensitivity, while symmetric zero-point-free arithmetic simplifies the datapath.

B. Real-Digit Integer-Reference Results

The host integer-reference pipeline correctly classifies 98 of 100 photographed digits. The host preprocessing path and integer pipeline agree on all predictions. The errors are sample 64, a 6 classified as 5 with confidence 51, and sample 93, a 9 classified as 3 with confidence 60. Classes 6 and 9 reach 9/10; all other classes reach 10/10.

The error pattern is consistent with shape ambiguity after thresholding: an open loop can make 6 resemble 5, while a distorted lower stroke can make 9 resemble 3. Confidence is diagnostic rather than calibrated probability. Sixty-six samples have confidence 100, 24 fall in 90–99, five in 70–89, and five in 50–69. The small, balanced, and likely correlated set makes 98/100 an engineering regression result, not an estimate of real-world generalization.

C. Golden Arithmetic and Full-System Simulation

Both convolution layers are checked element by element against PyTorch-derived INT32 outputs. Conv1 uses 912 activation bytes, 272 weight bytes, and 50,304 golden-output bytes. Conv2 uses 3,264 activation bytes, 4,736 weight bytes, and 25,216 golden-output bytes. Complete-array equality is stronger than checking only the final class.

The full-system ModelSim run exercises the CPU, UART bootloader, SD-SPI model, firmware, and accelerator. It ends with the recorded message `RV32I C FULL CNN FROM SD PASSED prediction=0`, zero errors, and one warning caused by an extra preload word. Simulated time is 6,430,982,150 ns; host wall-clock duration is 1:46:17. The latter is event-driven simulation cost, not inference latency. Because the transcript contains one final passing case, it is an integration regression rather than a statistical accuracy test.

TABLE V
POST-PLACE-AND-ROUTE UTILIZATION

Resource	Used	Total	Util.
Logic	15,715	20,736	76%
Registers	3,264	15,750	21%
CLS	8,870	10,368	86%
I/O ports	26	66	40%
BSRAM	46	46	100%
DSP	22	24	92%

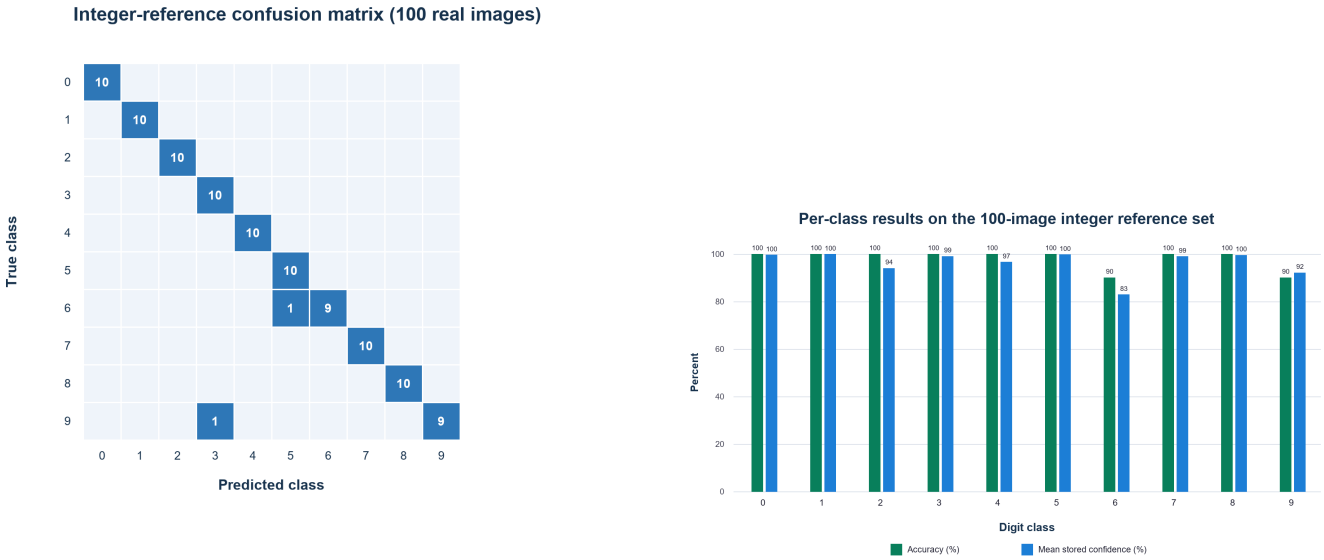
D. Resource Utilization

Fig. 5 and Table V show the final implementation. Logic reaches 76%, configurable logic sections 86%, block SRAM 100%, and DSP 92%. The design fits but has little headroom for larger buffers or more PEs.

The 22 DSP blocks do not imply exactly one block per PE: the primitive report includes multiple multiplier and multiply-add forms, and other arithmetic can consume DSP resources. The defensible conclusion is that both BRAM and DSP capacity are near saturation. Future performance work should prioritize reuse and scheduling before increasing parallelism.

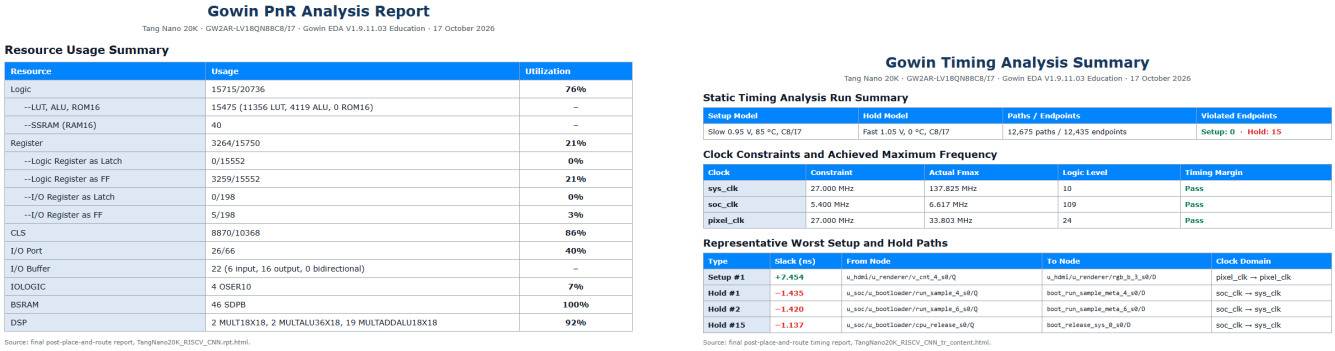
E. Timing Analysis

The 27 MHz system clock has a reported maximum frequency of 137.826 MHz, the 5.4 MHz SoC clock 6.617 MHz, and the 27 MHz pixel clock 33.804 MHz. Setup checks pass with worst setup slack +7.454 ns. However, 15 endpoints violate hold timing; worst hold slack is -1.435 ns. Representative



(a) (b)

Fig. 4. Host integer-reference results on 100 real digits: (a) confusion matrix and (b) per-class accuracy with confidence distribution.



Gowin PnR Analysis Report

Tang Nano 20K · GW2AR-LV18QN88C8/I7 · Gowin EDA V1.9.11.03 Education · 17 October 2026

Resource Usage Summary

Resource	Usage	Utilization
Logic	15715/20736	76%
--LUT, ALU, ROM16	15475 (11356 LUT, 4119 ALU, 0 ROM16)	–
--SSRAM (RAM16)	40	–
Register	3264/15750	21%
--Logic Register as Latch	0/15552	0%
--Logic Register as FF	3259/15552	21%
--I/O Register as Latch	0/198	0%
--I/O Register as FF	5/198	3%
CLS	8870/10368	86%
I/O Port	26/66	40%
I/O Buffer	22 (6 input, 16 output, 0 bidirectional)	–
IOLOGIC	4 OSER10	7%
BSRAM	46 SDPB	100%
DSP	2 MULT18X18, 2 MULTALU36X18, 19 MULTADDALU18X18	92%

Source: final post-place-and-route report, TangNano20K_RISCV_CNN.rpt.html.

Gowin Timing Analysis Summary

Tang Nano 20K · GW2AR-LV18QN88C8/I7 · Gowin EDA V1.9.11.03 Education · 17 October 2026

Static Timing Analysis Run Summary

Setup Model	Hold Model	Paths / Endpoints	Violated Endpoints
Slow 0.95 V, 85 °C, C8/I7	Fast 1.05 V, 0 °C, C8/I7	12,675 paths / 12,435 endpoints	Setup: 0 · Hold: 15

Clock Constraints and Achieved Maximum Frequency

Clock	Constraint	Actual Fmax	Logic Level	Timing Margin
sys_clk	27.000 MHz	137.825 MHz	10	Pass
soc_clk	5.400 MHz	6.617 MHz	109	Pass
pixel_clk	27.000 MHz	33.803 MHz	24	Pass

Representative Worst Setup and Hold Paths

Type	Slack (ns)	From Node	To Node	Clock Domain
Setup #1	+7.454	u_hdmi/u_renderer/v_cnt_4_s0/Q	u_hdmi/u_renderer/rgb_b_3_s0/D	pixel_clk → pixel_clk
Hold #1	-1.435	u_soc/u_bootloader/run_sample_4_s0/Q	boot_run_sample_meta_4_s0/D	soc_clk → sys_clk
Hold #2	-1.420	u_soc/u_bootloader/run_sample_6_s0/Q	boot_run_sample_meta_6_s0/D	soc_clk → sys_clk
Hold #15	-1.137	u_soc/u_bootloader/cpu_release_s0/Q	boot_release_sys_0_s0/D	soc_clk → sys_clk

Source: final post-place-and-route timing report, TangNano20K_RISCV_CNN_tr_content.html.

(a) (b)

Fig. 5. Gowin post-P&R evidence dated 17 October 2026: (a) resource summary and (b) static timing summary.

paths cross from the SoC clock into the system clock domain around run-sample and boot-release controls.

This pattern indicates a clock-domain-crossing or constraint issue rather than a long convolution datapath. Setup margin cannot compensate for negative hold slack. Remediation should use synchronizers or request-acknowledge handshakes, stabilize multi-bit buses around control pulses, correct clock relationships, and repeat P&R until both setup and hold violations are zero.

F. Board-Level Observations

Fig. 6 confirms that the board is connected, the GUI reaches the COM interface, and HDMI displays an input and prediction. One image shows class 6 with confidence 100%. Other observations include correct examples, a 9 classified as 3, and an out-of-dataset image shown as 8 with confidence 85%.

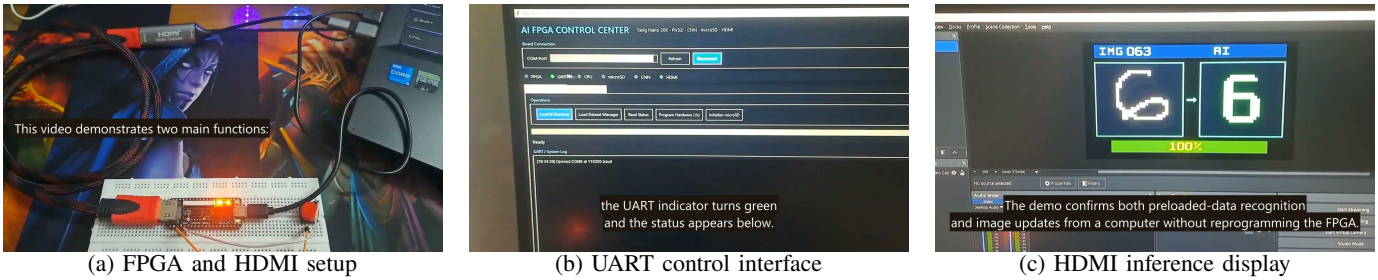
The photographs do not simultaneously record artifact hashes, labels, full UART logs, and cycle counts. They establish operational plausibility, not board-wide accuracy. Combining the host 98/100 result with visual board evidence would exceed the available data.

G. Comparison with the Reference Study

The studies are complementary. The reference provides a broader architectural search space; this work exposes implementation cost and integration faults. Their cosine similarity and simulator times cannot be numerically compared with MNIST accuracy or the present simulation duration.

H. Threats to Validity

Internal validity is limited because simulated INT8 has not been proven identical to every firmware fixed-point operation and the full-system transcript contains only one final case. External validity is limited by the small, controlled 100-image set. Measurement validity is limited by uncalibrated confidence, missing stage cycle counts, and no software-only latency baseline. Implementation validity is limited by 15 hold-violating endpoints. Reproducibility is limited because board photographs are not bound to a manifest of bitstream, firmware, SD-image hashes, and a complete 100-sample log.



(a) FPGA and HDMI setup (b) UART control interface (c) HDMI inference display

Fig. 6. Board-level operational evidence. The images demonstrate integration and observability, not a controlled 100-sample accuracy or latency experiment.

TABLE VI
EVIDENCE-BASED COMPARISON WITH LIM *ET AL.* [3]

Criterion	Lim *et al.*	This work
Workload	Darknet/YOLO-tiny-v3 representative layers	End-to-end LeNet-inspired 28-by-28 classifier
Numerical result	Layer-output cosine similarity approximately 85% or higher	99.50% MNIST Float32/INT8; 98/100 host real-digit integer pipeline
Execution result	7.62–107.91 s simulator wall-clock	1:46:17 full-system simulator wall-clock for one case; no board latency
Design space	PE count, 1–8 MB buffers, precision, and layer behavior	One 16-PE point with 3,136-byte input, 4,608-byte weight, and 12,544-word output buffers
Physical evidence	No DLA synthesis/P&R report	Logic 76%, BSRAM 100%, DSP 92%; setup passes, 15 hold violations
System scope	Virtual CPU/accelerator and ELF/Darknet flow	Flash, UART, firmware, microSD, CRC, GUI, HDMI, synthesized RTL

A controlled board experiment should freeze and hash all artifacts, execute the same 100 records without intervention, retain every failure in the denominator, and record labels, predictions, logits or confidence, error status, and stage cycle counts. Timing closure should reach zero setup and zero hold violations before voltage or temperature claims. Only then should board-wide accuracy, latency, speedup, or energy be reported.

VI. CONCLUSION

This paper presented a complete RISC-V-controlled INT8 CNN system for the Tang Nano 20K, spanning training, quantization, golden-model generation, RTL acceleration, firmware, storage, host control, HDMI, and physical implementation. The model reaches 99.50% in both Float32 and simulated INT8 MNIST evaluation; the separate host integer-reference path reaches 98/100 on a balanced real-digit set. Element-wise convolution checks and full-system simulation connect the notebook to RTL, while P&R confirms that the design fits the device.

The evidence also defines the current boundary: BSRAM and DSP are near capacity, 15 hold violations remain, and neither a version-matched board regression nor end-to-end latency is available. The principal contribution is therefore an evidence-traceable hardware-software realization, not an unsupported claim of superior accuracy or speed.

Future work should close clock-domain and hold timing, add cycle counters and reproducible board regression, then investigate double buffering, streaming weights, and improved scheduling. A software-only RV32I baseline and board power measurement would enable defensible speedup and energy comparisons. Broader handwriting data and preprocessing stress tests are required to evaluate generalization.

REFERENCES

[1] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.

[2] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.

[3] S.-H. Lim, W. W. Suh, J.-Y. Kim, and S.-Y. Cho, "Risc-v virtual platform-based convolutional neural network accelerator implemented in systemc," *Electronics*, vol. 10, no. 13, p. 1514, 2021.

[4] *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, RISC-V International, 2024. [Online]. Available: https://docs.riscv.org/reference/isa/unpriv/rv32.html

[5] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.

[6] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proc. International Conference on Machine Learning*, 2010, pp. 807–814.

[7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[8] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[9] Y. LeCun, C. Cortes, and C. J. C. Burges, "The mnist database of handwritten digits," https://yann.lecun.com/exdb/mnist/, accessed: Oct. 2026.

[10] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.